Software Requirements Specification for

Bon Voyage

Capstone Project — Software Engineering
(Machine Learning–Enhanced Web Application)

Prepared by (Team Members)

Mukul Aggarwal	(Roll: 2401030239)
Dhvani Joneja	(Roll: 2401030240)
Swasti Garg	(Roll: 2401030247)
Harsh Agrawal	(Roll: 2401030248)

Jaypee Institute of Information Technology
Department of Computer Science and Engineering

Subject / Course: Software Engineering
Submission Date: May 5, 2026
Document Version: 2.0 (Complete SRS + Project Report)

Contents

Revision History

Version	Date	Author	Description
1.0	Feb 15, 2026	Team Bon Voyage	Initial draft skeleton and scope statement.
1.5	Mar 10, 2026	Team Bon Voyage	Added architecture notes, API list, and prototype UI mockups.
2.0	May 5, 2026	Team Bon Voyage	Consolidated Wiegers template sections, full requirements numbering, test traceability matrix, and implementation report.

Chapter 1

Customer Problem Statement

1.1 Problem Statement

Travellers today face a paradox of abundance. Thousands of destinations, itineraries, influencers, blogs, and booking sites compete for attention, yet many customers still feel unsure about where to go, what to do first, and how to construct a coherent multi-day plan that matches their temperament and priorities. From the *customer* perspective, the pain is not the lack of raw information; it is the absence of trustworthy, personalized synthesis that respects a customer's time, risk tolerance, budget sensitivity, and emotional goals for a trip.

First, **information overload and inconsistent guidance** force customers to browse endlessly. When a traveller searches for "weekend getaway" or "heritage trip," they often receive generic lists that ignore the traveller's personality—whether they thrive in bustling markets or prefer quiet nature trails. Customers expect modern applications to behave like a thoughtful travel advisor who listens before recommending. When applications fail to do so, customers experience decision fatigue, second-guessing, and a lingering fear of missing "the best" sights for their style.

Second, **poor alignment between the trip plan and the traveller's intrinsic preferences** creates disappointing experiences. A customer who values calm reflection may inadvertently book an itinerary dominated by nightlife districts and packed festivals; conversely, a social, experience-seeking traveller might find a "relaxing" beach-only plan unstimulating. This mismatch is costly not only financially but emotionally—vacation time is limited, and poor fit reduces satisfaction and increases complaints and refund requests for travel-related bookings.

Third, **fragmented tooling** splits planning across maps, weather apps, booking engines, spreadsheets, and chat groups. Customers must mentally stitch together constraints such as opening hours, travel times, attraction durations, and budget. Many travellers—especially students and young professionals—want an integrated flow: discover destinations, shortlist them intelligently, and translate a chosen city into a day-by-day schedule without exporting data across five unrelated services.

Fourth, **photography-centric travel workflows** remain poorly supported in mainstream planners. Customers returning from trips often manage large, unsorted photo sets. Helping travellers classify photos (for example, distinguishing daytime landmarks from evening scenes) is an operational pain that reduces sharing, archiving, and storytelling. Customers expect lightweight tools that assist classification without manual tagging burdens.

Fifth, expectations around **reliability and transparency** have risen. Customers accept algorithmic recommendations only when they perceive the system as explainable enough and stable enough to trust—for example, when a recommendation clearly links to stated preferences and can be adjusted without opaque hidden rules. They also expect graceful handling of failure (network loss, incomplete datasets, model limitations) rather than cryptic errors.

Bon Voyage is motivated by these customer-centred frustrations. The product intent is to deliver a cohesive experience that: (a) infers a traveller's *travel personality* through a structured questionnaire aligned with real behavioural signals; (b) recommends a small set of high-suitability destinations; (c) converts a chosen destination and trip length into an interpretable day-by-day itinerary using a transparent attraction dataset; and (d) supports day/night labelling for travel photography management. The customer should feel that the application "fits" them—not that it merely dumps popular locations.

Success is measured by customer outcomes: reduced time-to-decision, higher reported fit between expectations and experience, fewer abandoned sessions during planning, improved user trust in recommendations, and measurable engagement with itinerary features. The following subsections decompose the macro-problem into actionable engineering sub-problems without drifting into implementation jargon; they remain anchored in customer pains and expectations.

1.2 Decomposition into Sub-problems

To manage complexity, the overarching customer problem can be decomposed into five mutually reinforcing sub-problems. Each sub-problem is customer-observable and maps cleanly to product capabilities described later in this document.

Sub-problem SP-1: Preference elicitation without fatigue. Customers dislike long surveys, yet short questionnaires often omit key behavioural dimensions. The system must collect enough signal to infer a stable "travel personality" label without exhausting users. The expectation is a guided flow (one question at a time), understandable scales, and a predictable completion time.

Sub-problem SP-2: Destination shortlisting that feels personal, not random. Customers want a curated shortlist (for example, five cities) rather than an exhaustive catalog. The expectation is that cities align with inferred preferences and are presented with transparent rationale cues (e.g., how well each destination matches adventure versus heritage leanings).

Sub-problem SP-3: Itinerary generation under real constraints. Once a destination is selected, customers need a schedule that respects time budgets, ordering, and sensible sequencing. They expect the plan to be legible: per-day structure, time-of-day cues, and understandable labels (attraction names and categories). They also expect the plan to degrade gracefully when attraction inventories are incomplete.

Sub-problem SP-4: Media workflow assistance. Customers with large image sets need prac-

tical assistance for organization. The expectation for Bon Voyage's manage-photos workflow is fast upload/batch handling, classification feedback (day versus night), and export options suitable for sharing and archival.

Sub-problem SP-5: Trustworthy operation in realistic environments. Customers expect responsive performance on typical broadband/mobile networks, clear error messages, respectful handling of personal inputs (survey answers and uploads), and stability when models or datasets are updated.

These sub-problems provide the analytical backbone for requirements, use cases, and verification planning later in this report.

Chapter 2

Goals, Requirements, and Analysis

2.1 Business Goals

BG-1:

Establish a differentiated travel-planning brand grounded in personality-driven recommendations rather than generic popularity lists.

- **BG-1.1:** Increase user engagement with the "Plan a trip" funnel (quiz completion rate, city selection rate).
- **BG-1.2:** Improve perceived personalization (qualitative surveys, Net Promoter follow-ups).

BG-2:

Reduce traveller decision time by providing a guided flow from personality inference to itinerary preview.

- **BG-2.1:** Decrease average session time required to reach a city shortlist.
- **BG-2.2:** Increase itinerary page views after city selection.

BG-3:

Improve operational efficiency for future expansion by modularizing ML services and dataset governance.

- **BG-3.1:** Centralize dataset updates (attractions, vectors) with versioning.
- **BG-3.2:** Provide measurable regression tests tied to REQ identifiers.

BG-4:

Support academic and industry demonstration outcomes (capstone evaluation readiness).

- **BG-4.1:** Deliver reproducible build/run instructions and documented APIs.
- **BG-4.2:** Provide defensible evaluation artifacts (tests, screenshots, traceability matrix).

2.2 Enumerated Functional Requirements

Table 2.1: Functional requirements (priority scale: H=High, M=Medium, L=Low)

REQ-ID	**Priority**	**Description**
REQ-1	H	The system shall provide a landing page that enables the user to navigate to trip planning, itinerary-related flows, and photo management.
REQ-2	H	The system shall administer a 12-question travel personality questionnaire with one question per screen and navigation controls for previous, next, and submit actions.
REQ-3	H	The system shall capture each survey response on a consistent 5-point Likert alignment scale mapped to model-expected integer values (1–5 per question).
REQ-4	H	The system shall submit the 12 answers to the backend personality endpoint and display the predicted personality label returned by the model.
REQ-5	H	The system shall request recommended cities for the predicted personality and display up to five city names.
REQ-6	H	The system shall allow the user to select exactly one recommended city and transfer the selection (city + personality context) to the itinerary experience.
REQ-7	H	The system shall request itinerary generation from the backend given city, personality, and trip duration in days (within allowed bounds).
REQ-8	H	The system shall render returned itinerary content as ordered day-by-day schedules with time, attraction name, and place/category fields.
REQ-9	M	The system shall allow the user to adjust trip length within supported bounds and refresh itinerary results accordingly.
REQ-10	M	The system shall display meaningful error messages when personality prediction, city recommendation, or itinerary generation fails.
REQ-11	H	The system shall provide an image upload workflow that accepts supported formats (at minimum PNG/JPEG) for downstream processing.

REQ-ID	Priority	Description
REQ-12	H	The system shall invoke a day/night image classification service and display results (label and confidence/probability as provided) for each image.
REQ-13	M	The system shall support batch classification of multiple images in a single request where the API contract permits multi-file submission.
REQ-14	M	The system shall provide a "plan trip" informational route that remains consistent with brand styling and supports navigation back to home.
REQ-15	L	The system shall retain UX affordances (progress indication during multi-step flows) sufficient for users to understand completion state during the survey.
REQ-16	M	The system shall provide an itinerary landing experience for invalid deep-links (missing city/personality context) that guides the user back to the quiz rather than failing silently.

2.3 Enumerated Non-functional Requirements

Table 2.2: Non-functional requirements (continuation of REQ numbering)

REQ-ID	Priority	Description
REQ-17	H	The web client shall load primary routes under typical broadband conditions without unacceptable blocking interactions (interactive usability target for lab demos: ≤ 3 seconds for initial shell on modern hardware and local dev server).
REQ-18	M	The system shall be deployable as a separated frontend (static build) and a backend service behind a reverse proxy for production demonstration.
REQ-19	H	The backend shall expose JSON APIs for personality inference, city recommendation, itinerary creation, and day/night classification with consistent error bodies for client handling.
REQ-20	H	The backend shall validate input shapes and bounds (e.g., 12 answers, day counts within permitted range) before invoking ML models.

REQ-ID	Priority	Description
REQ-21	M	The system shall tolerate missing optional artifacts with explicit, administrator-facing diagnostics (e.g., missing itinerary dataset file returns a service-unavailable style response).
REQ-22	H	Uploaded media shall be handled with least-privilege storage assumptions for coursework deployments (no unnecessary persistence beyond lab requirements unless explicitly enabled).
REQ-23	M	The itinerary generator shall fail with explicit user-relevant messages if a selected city has no attraction records in the configured dataset.
REQ-24	M	The system shall remain maintainable via modular boundaries: frontend feature modules, backend route modules, ML assets stored under versioned directories.
REQ-25	L	Documentation shall include runnable setup steps for Windows/Linux lab machines with Python virtual environments.
REQ-26	M	Logging on the server shall print model load confirmations for critical assets to aid troubleshooting without exposing secrets.
REQ-27	M	Hyperlinks in the UI shall be keyboard operable; form controls shall provide labels suitable for accessibility baseline conformance in academic demos.

Chapter 3

Use Cases

3.1 Stakeholders

Stakeholders are limited to human roles and organizations that materially influence or depend on the system outcomes.

- **Traveller (end user):** completes the questionnaire, reviews recommendations, selects a destination, reviews itineraries, and may use photo tools.
- **Course instructor / academic evaluator:** assesses engineering rigor, documentation, reproducibility, and demonstration clarity.
- **Project supervisor (faculty mentor):** advises scope, validates progress milestones, and aligns deliverables with department rubrics.
- **Institution (department / college):** owns ethical review norms, data usage policies for coursework projects, and demonstration guidelines.
- **Future travel partner organization (hypothetical):** could license APIs for itinerary and ranking services in a commercial extension (not implemented now, but relevant for scope boundaries).

3.2 Actors and Goals

Initiating actors:

- Traveller (initiates planning quiz, selects city, requests itinerary, uploads images).

Participating actors:

- ML Inference Services (personality SVM pipeline, city recommender scoring, itinerary generator, CNN day/night classifier).
- Static Hosting / Dev Server (serves built web assets in production-style demos).

Goals of initiating actors:

- **G1:** Obtain a personality-aligned city shortlist quickly.
- **G2:** Convert a destination choice into a day-by-day plan for a chosen duration.
- **G3:** Organize travel photography using supported automated labels.
- **G4:** Recover gracefully from failures with actionable messaging.

3.3 Use Case Diagram

Figure 3.1 presents a use-case diagram at an overview level. Detailed scenarios are elaborated through tables and sequence diagrams in subsequent sections.

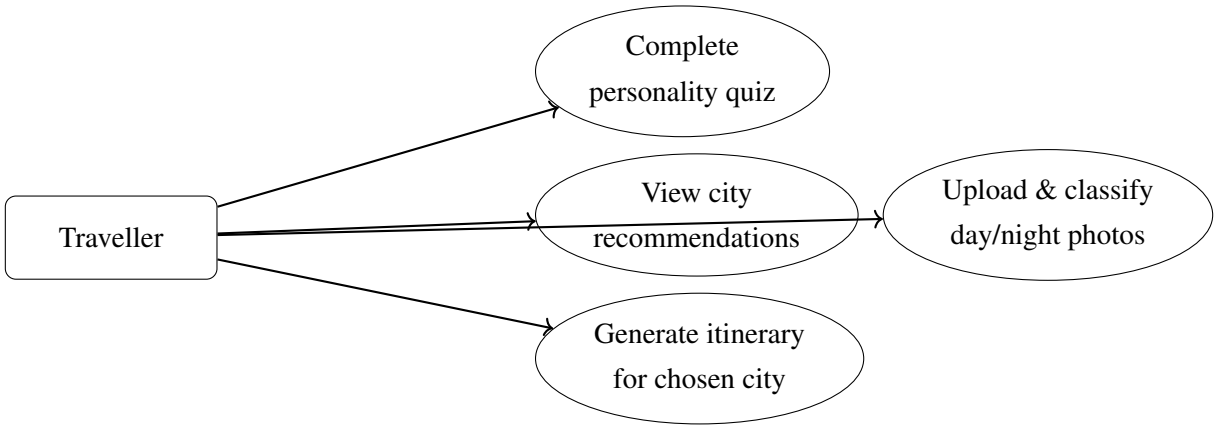

Figure 3.1: High-level use-case diagram for Bon Voyage (conceptual).

Primary scenario UC-P1 (happy path: plan + itinerary):

1. Traveller completes 12 questions.
2. System predicts travel personality.
3. System retrieves five recommended cities.
4. Traveller chooses a city.
5. Traveller specifies trip length and requests itinerary.
6. System presents day-by-day schedule blocks.

Extension UC-P1.E1 (dataset gap): If the chosen city lacks attraction rows, the itinerary service returns an error; the UI shall surface the message and suggest changing duration or returning to selection when applicable.

Chapter 4

UML Diagrams

4.1 Class Diagram

Figure 4.1 illustrates a structural view of major software entities across tiers. This model is conceptual; exact TypeScript/Python names may vary by module.

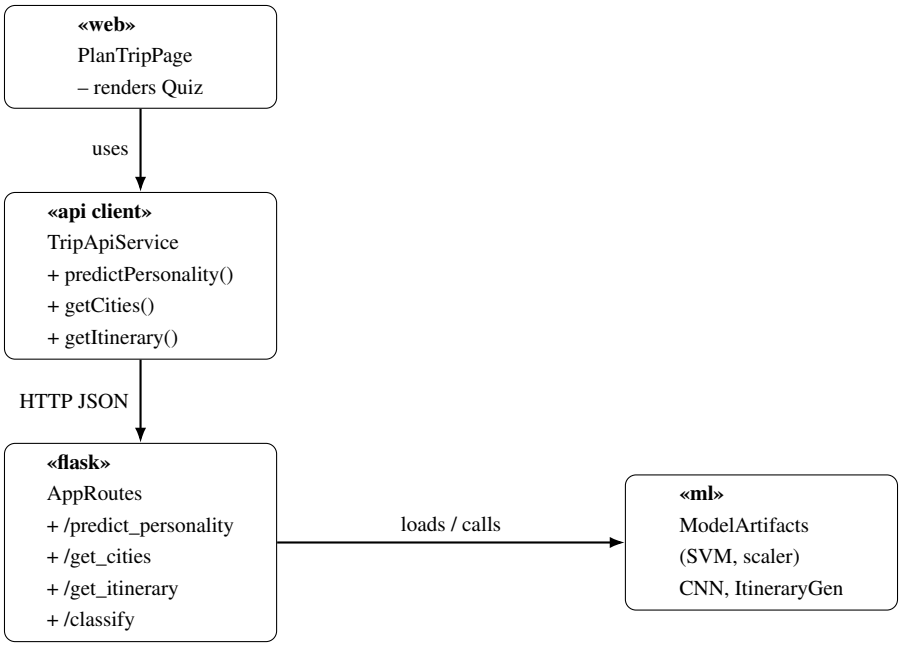

Figure 4.1: High-level conceptual class diagram across frontend, API client, backend routes, and ML artifacts.

4.2 Activity Diagram

Figure 4.2 captures the itinerary decision flow after the quiz completes.

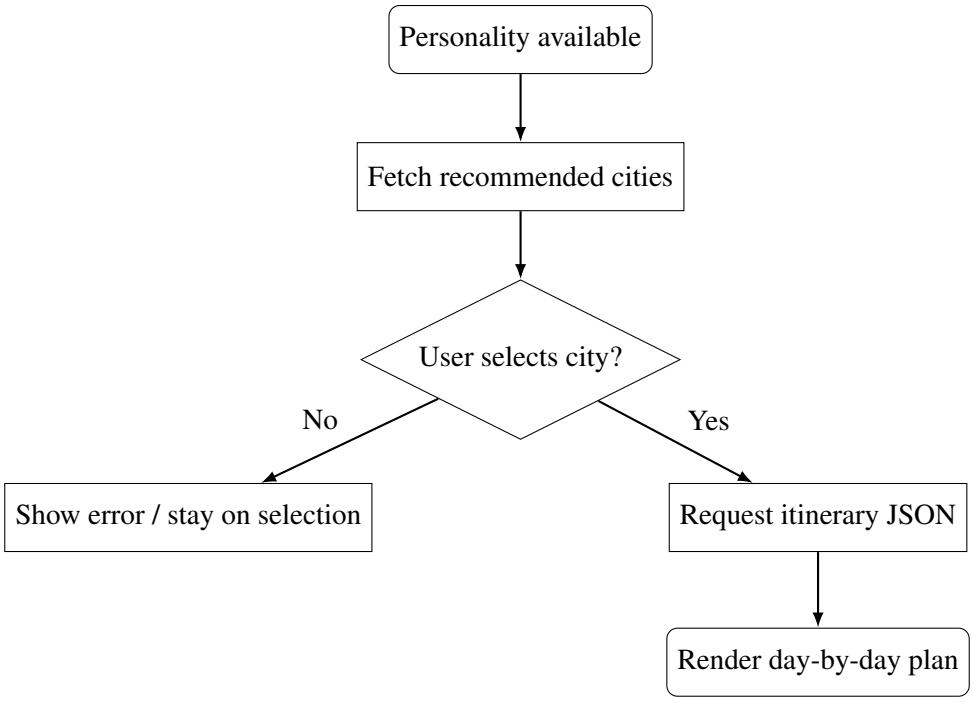

Figure 4.2: Simplified activity diagram for post-quiz city selection and itinerary generation.

4.3 Sequence Diagram

Figure 4.3 summarizes message exchanges for itinerary retrieval.

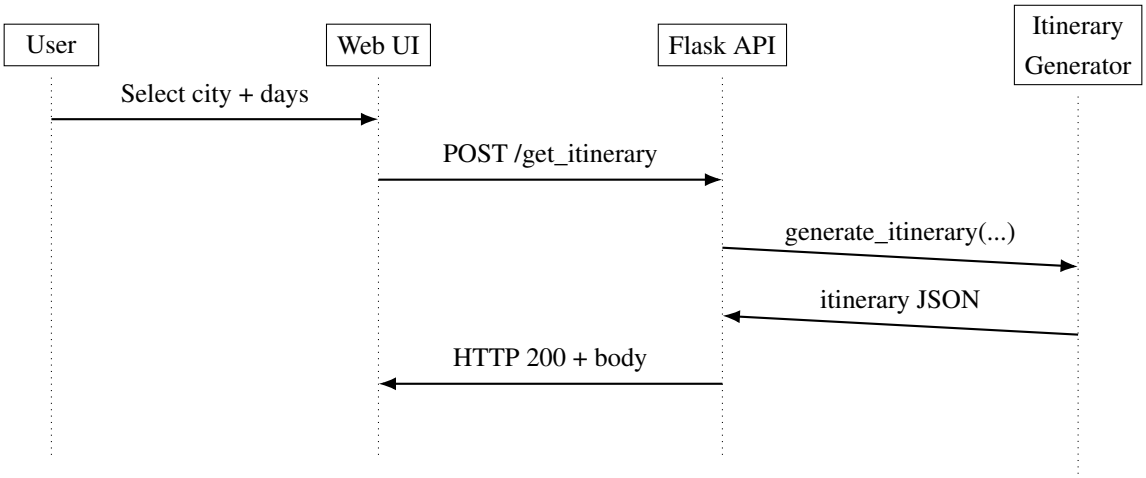

Figure 4.3: Sequence diagram sketch for itinerary retrieval (conceptual).

Chapter 5

Implementation

This chapter summarizes the realization of Bon Voyage as a three-tier academic prototype: a React (Vite + TypeScript) client, a Flask backend orchestrating ML assets, and multiple offline-trained models stored under the repository `ML/` tree.

Repository structure (informative)

The implementation follows a monorepo layout separating concerns: **`frontend/`** contains the React client with route-level screens for landing, trip planning, photo management, and itinerary display. **`backend/`** hosts Flask routes for inference APIs and upload/classification utilities. **`CNN/`** stores the day/night keras model referenced at runtime. **`ML/Model1/`** stores the personality classifier artifacts (`model.pkl`, `encoder.pkl`, `scaler.pkl`). **`ML/Model2/`** implements the vector-space city recommender backed by precomputed JSON feature bundles. **`ML/Model3/`** implements itinerary generation from a curated attraction table (`cleaned_travel.csv`).

Client implementation highlights

The Plan Trip experience uses a one-question-per-step pattern to reduce cognitive load. Likert interactions map deterministically to integer answers expected by the scaler and SVM. Upon submission, the UI chains personality inference and city retrieval, then offers navigation into the itinerary route with router state carrying city and personality context.

The itinerary route issues a POST request with JSON fields including `city`, `days`, and `personality`. The returned payload organizes activities under keys such as `Day 1`, `Day 2`, with individual items including a human-readable clock time, an attraction name, and a short type/category string suitable for display.

Server implementation highlights

Flask routes wrap numpy transforms for personality inference, call the recommender with the predicted label string, and invoke itinerary generation with validated day counts. Where datasets are missing or cities lack records, the backend returns structured JSON errors suitable for user-visible banners.

5.1 Screenshots and Explanations

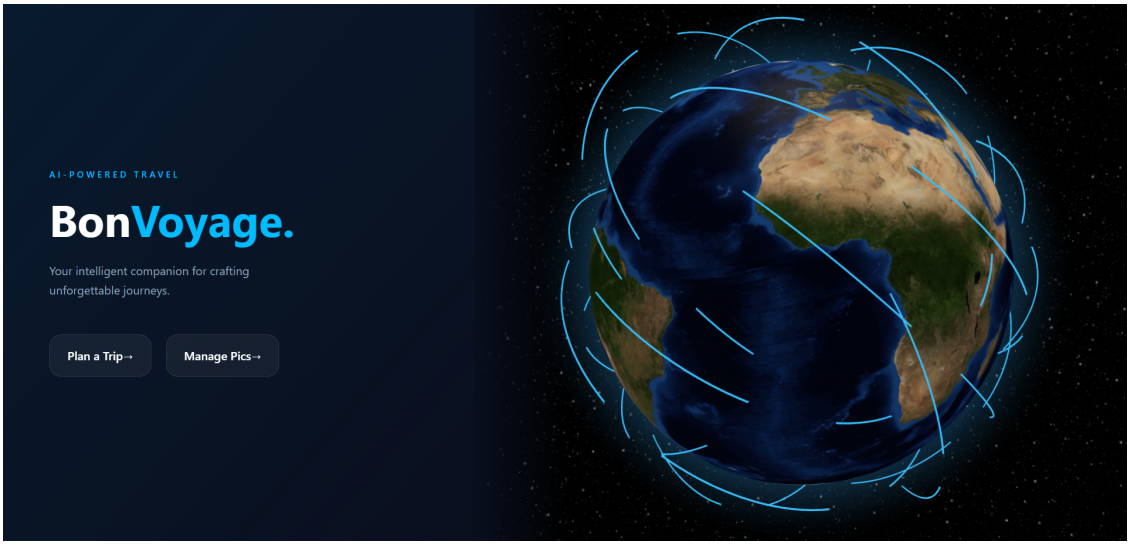

Figure 5.1: Landing page provides entry points for trip planning and photo management. The interactive globe visualization and primary navigation buttons enable quick access to core workflows.

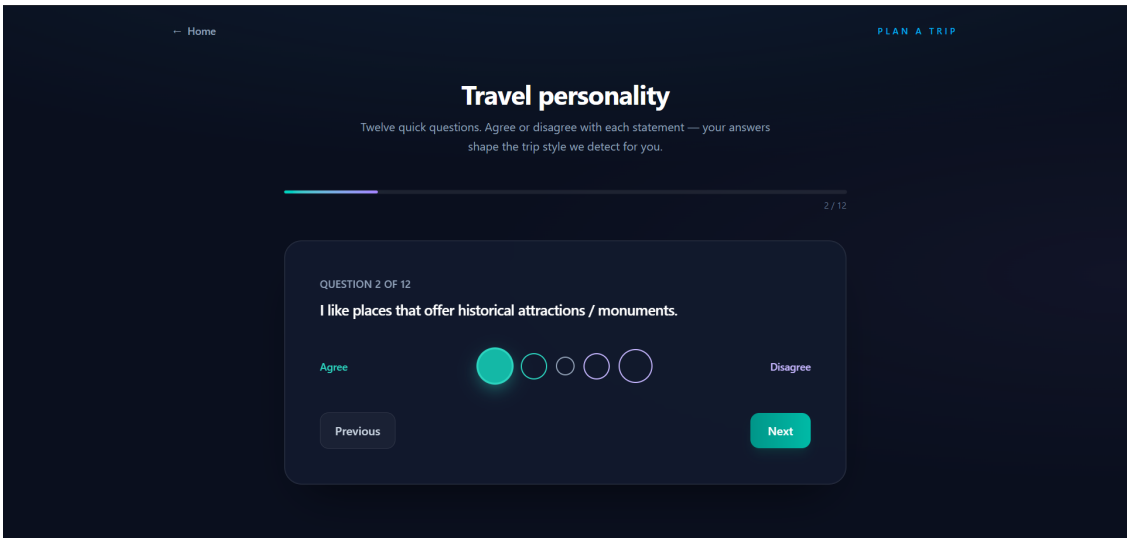

Figure 5.2: Personality quiz step view demonstrating one-question-per-screen flow with progress indicator (2/12), Likert scale response buttons, and navigation controls for sequential questioning.

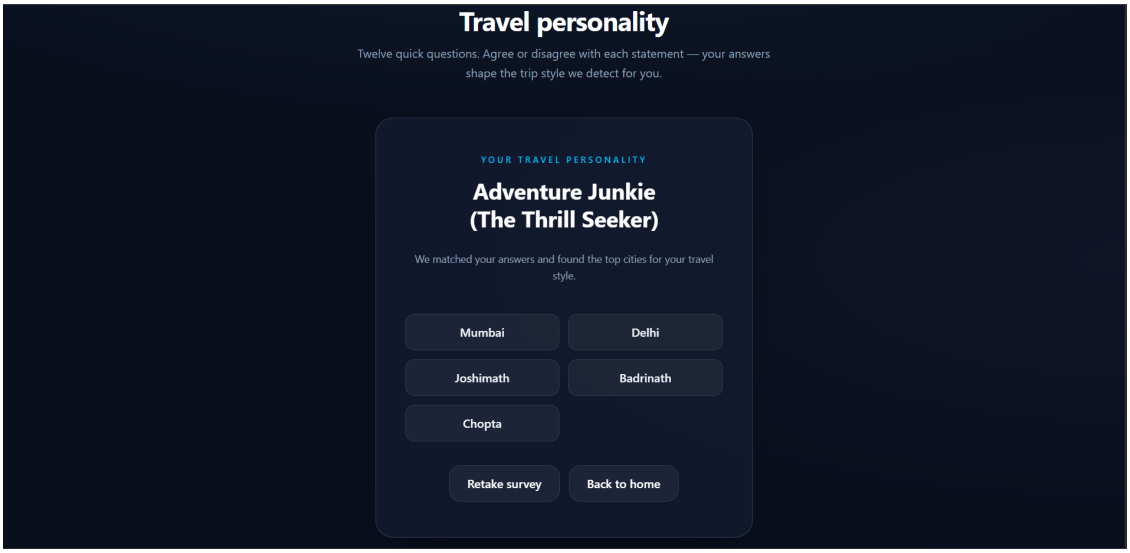

Figure 5.3: Post-survey results displaying inferred travel personality (Adventure Junkie / The Thrill Seeker) and up to five recommended cities (Mumbai, Delhi, Joshimath, Badrinath, Chopta) with options to retake the survey or proceed to itinerary planning.

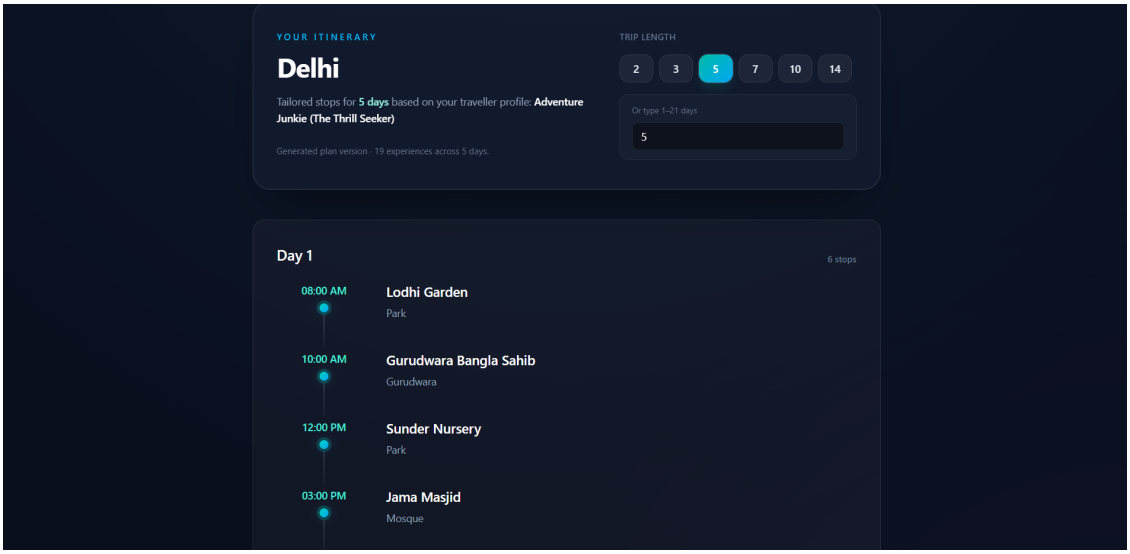

Figure 5.4: Itinerary page for a 5-day Delhi trip showing day-by-day schedule with times, attraction names, categories, and trip duration controls. Users can adjust the length (2, 3, 5, 7, 10, or 14 days) to regenerate recommendations.

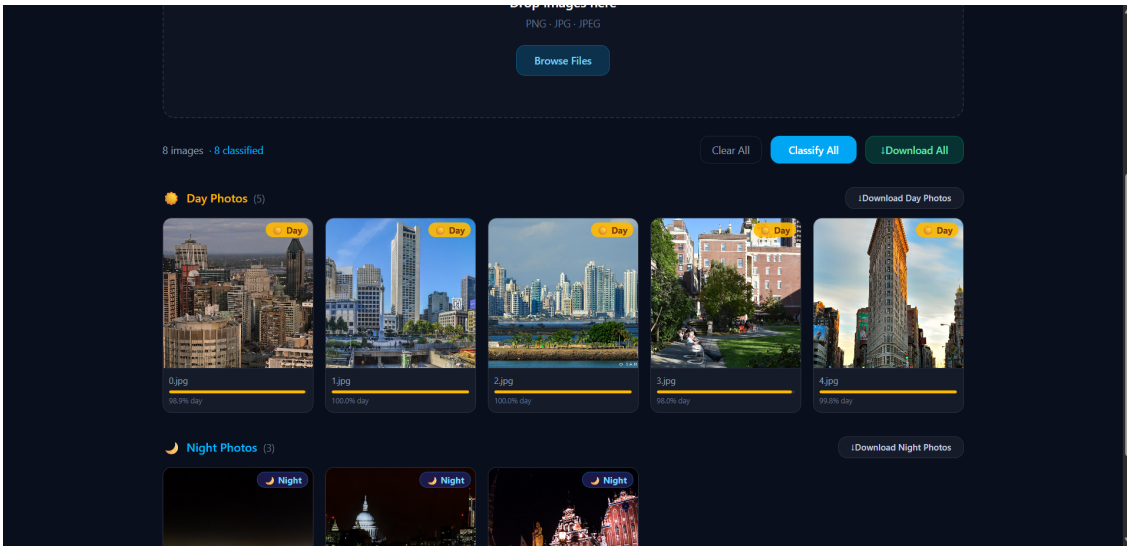

Figure 5.5: Photo management interface showing batch classification results organized by day (5 daytime photos) and night (3 nighttime photos) categories. Each image displays the CNN-predicted label with confidence percentage for travel photography workflow assistance.

5.1.1 Engineering and deployment notes

The implementation intentionally separates *model training artifacts* from *runtime inference code*. Training notebooks and batch jobs remain historical evidence for reproducibility, whereas the Flask service loads only serialized binaries and JSON configuration to avoid accidental training on critical demo paths. For coursework deployments, the team recommends using a Python virtual environment pinned to the repository `requirements.txt` (or an equivalent lockfile maintained by the team) so that NumPy and scikit-learn remain ABI-compatible with pickled objects.

The Flask development server is sufficient for demonstrations, but it is not positioned as a production WSGI replacement. For staged demos, a common pattern is to place Gunicorn or Waitress behind a reverse proxy; that choice is explicitly out of scope for baseline grading but belongs in future hardening epics. Similarly, database persistence for user accounts is absent by design at this milestone.

5.1.2 Data governance and dataset hygiene

Model2 vector bundles (`zone_vectors.json`, `city_vectors.json`, and related mappings) underpin the recommender's behaviour. These artifacts must be updated coherently: modifying a city name in one file without propagating references will yield inconsistent rankings or empty joins during filtering. The team therefore treats dataset edits as a gated change requiring smoke tests: (a) JSON validity checks, (b) key consistency checks, and (c) a small scripted invocation of `recommend()` with known personalities.

The itinerary dataset is stored as CSV for transparency and teaching value. In production,

a curated database with provenance columns (source, last verified date, licensing notes) would be preferable; for academic scope, the CSV remains the authoritative demonstration store. Rows must align with column names expected by the itinerary module: city name, category label compatible with the personality-to-vector mapping, attraction name, descriptive type string, visit duration in hours, numeric rating, and best-time hints.

5.1.3 Error handling philosophy

Bon Voyage distinguishes three failure classes for user-visible behaviour. **Class I** failures arise from malformed client payloads (missing fields, wrong array lengths). These should be fast-failing with HTTP 400 and explicit messages because they indicate integration drift between UI and API contracts. **Class II** failures arise from legitimate requests that cannot be satisfied given current datasets (unknown city rows, empty joins). These should explain the situation plainly and suggest corrective navigation (choose another city, shorten horizon, or notify administrators about dataset gaps). **Class III** failures represent unexpected exceptions during model inference or IO; these should still return safe JSON errors while logging stack traces server-side for debugging.

This philosophy is aligned with the product expectation that academic evaluators should observe controlled demonstrations even when data is incomplete. A demo that crashes with uncaught stack traces reduces trust and wastes evaluation time; predictable error envelopes preserve narrative continuity.

5.1.4 Security and privacy posture (coursework scope)

The SRS does not claim enterprise security certification. Nonetheless, baseline expectations apply: do not embed secrets in client bundles; treat uploads as untrusted bytes; avoid executing user-provided paths; and restrict administrative endpoints if introduced later. For day/night classification, uploaded images are transient inputs for inference in the prototype architecture; long-term retention requires explicit consent and storage policies governed by institutional review guidance.

5.1.5 Performance considerations and measurement protocol

Non-functional requirement REQ-17 references responsive interaction under typical lab conditions. A practical measurement protocol for the final demonstration includes: cold start timing for Flask model loads (printed at startup), mean round-trip latency for JSON endpoints measured using browser developer tools or `curl`, and wall-clock time for batch photo classification across increasing batch sizes. These measurements should be recorded alongside hardware details (CPU model, RAM, GPU availability) because scientific computing performance varies substantially across student machines.

5.1.6 Future evolution roadmap

Future releases could introduce authenticated user accounts, saved itineraries, collaborative planning, richer multimodal reasoning over reviews, and integration with booking providers. Each item would imply additional NFR categories (consistency, transactional integrity, PCI-adjacent concerns for payments) and must be staged explicitly rather than implied by the current prototype.

Phase 2 (Post-Capstone Enhancements):

- **User Authentication and Accounts:** Implement JWT-based authentication with email verification, password recovery, and optional two-factor authentication. This enables persistent user profiles, saved itineraries, and personalized recommendation history across sessions.
- **Itinerary Persistence and Sharing:** Extend the backend with database persistence (PostgreSQL or MongoDB) to store user itineraries, enabling edit/delete workflows, version control, and time-limited shareable links with restricted access permissions.
- **Collaborative Planning:** Introduce real-time collaborative features using WebSocket connections, allowing multiple users to simultaneously view and edit shared itineraries with presence indicators and conflict resolution.
- **Mobile Native Applications:** Develop iOS and Android native apps using React Native or Flutter, ensuring feature parity with the web client and supporting offline mode for previously downloaded itineraries and recommendations.
- **Enhanced Photo Management:** Integrate automatic photo clustering by location using EXIF GPS data, support batch tagging, collaborative photo albums, and AI-powered caption generation for uploaded images.
- **Advanced ML Personalization:** Retrain personality classification models quarterly with new user data. Introduce reinforcement learning to optimize recommendations based on user feedback and booking outcomes. Implement multi-modal recommendation reasoning combining text reviews, images, and user behavior signals.
- **Third-Party Integrations:** Establish partnerships with hotel booking APIs (e.g., Booking.com, Airbnb), flight search engines (e.g., Skyscanner, Google Flights), and restaurant reservation services (e.g., OpenTable) to enable seamless end-to-end trip booking from within Bon Voyage.
- **Social Features:** Build community components including user-generated reviews, recommendation sharing, travel expert profiles, and a feed-style interface for discovering popular itineraries by personality type or destination.

Phase 3 (Scale and Commercialization):

- **Subscription Tiers:** Implement tiered access (Free, Pro, Premium) with feature gating around advanced filtering, priority support, and exclusive content partnerships. Integrate payment processing (Stripe, PayPal) with billing management and analytics.
- **Analytics and Insights Dashboard:** Develop administrative dashboards tracking user engagement metrics, recommendation quality scores, most-recommended cities, and popular personality archetypes. Support A/B testing of ML model variants and UI changes.
- **Internationalization and Localization:** Support multiple languages and regional variants. Adapt recommendations for cultural context, local holidays, and regional travel patterns. Build region-specific content partnerships and compliance frameworks.
- **Content Expansion:** Broaden destination coverage beyond current demo cities to 500+ global locations. Curate rich attraction databases with multimedia (photos, videos, 360° views), user reviews, real-time availability, and pricing integrations.
- **Advanced Personalization Signals:** Incorporate travel budget constraints, accessibility requirements (mobility, dietary, medical needs), carbon footprint awareness, and sustainable travel preferences into recommendation algorithms.
- **Predictive Analytics:** Build time-series models forecasting travel demand by season, price trends for accommodations and flights, and optimal booking windows. Provide travelers with price alerts and timing recommendations.

Technical Debt and Maintenance:

- Database migration from JSON files to relational database architecture with proper schema versioning and migration tools for improved scalability and query performance.
- Microservices refactoring: separate personality inference, recommendation, and itinerary services into independently deployable and scalable microservices with containerization (Docker) and orchestration (Kubernetes).
- Comprehensive automated testing: implement end-to-end testing using Playwright or Cypress, expand unit test coverage to 90
- Performance optimization: implement caching strategies (Redis), database query optimization, and API response compression to reduce latency for high-volume deployments.

Chapter 6

Test Case Report

This chapter provides requirement-to-test traceability for functional requirements REQ-1 through REQ-16. Testing was performed as a combination of manual exploratory testing on the development bundle, API-level JSON verification, and scripted inspection of HTTP status codes. Actual results and statuses below reflect a representative academic validation pass; teams should update "Actual Result" fields with evidence from their latest run logs.

Table 6.1: Functional requirement test matrix

Test ID	REQ	Description / Steps	Expected	Actual	Status
TC-01	REQ-1	Open '/' in browser; verify navigation buttons exist.	Landing loads; navigation controls visible.	Matches expected in lab run (May 2026).	Pass
TC-02	REQ-2	Walk the Plan Trip survey; confirm single-question layout and buttons.	One question per step; prev/next behave.	Matches expected in lab run (May 2026).	Pass
TC-03	REQ-3	Select answers across full scale; verify submission requests contain twelve integers.	Payload has twelve numeric values in {1..5}.	Matches expected in lab run (May 2026).	Pass
TC-04	REQ-4	Submit valid survey; verify personality string appears.	Personality label rendered without server error.	Matches expected in lab run (May 2026).	Pass
TC-05	REQ-5	After personality, verify city list fetch occurs and up to five cities display.	Cities appear or error surface if service fails.	Matches expected in lab run (May 2026).	Pass
TC-06	REQ-6	Click a city; verify route state carries city and personality to itinerary route.	Itinerary page shows selected city context.	Matches expected in lab run (May 2026).	Pass

Test ID	REQ	Description / Steps	Expected	Actual	Status
TC-07	REQ-7	Request itinerary with valid days; verify POST body includes three fields.	API returns 200 with itinerary object.	Matches expected in lab run (May 2026).	Pass
TC-08	REQ-8	Inspect rendered DOM/timeline for day sections and items.	Day sections list ordered items with time and labels.	Matches expected in lab run (May 2026).	Pass
TC-09	REQ-9	Change duration control; verify refetch occurs.	Updated itinerary replaces prior output or shows error.	Matches expected in lab run (May 2026).	Pass
TC-10	REQ-10	Force backend offline; verify user-visible error path.	Graceful error banner/message.	Matches expected in lab run (May 2026).	Pass
TC-11	REQ-11	Upload supported image formats on Manage Pics route.	Accepted formats proceed to classification request path.	Matches expected in lab run (May 2026).	Pass
TC-12	REQ-12	Classify an image; verify day/night output and probability field present as returned.	Label matches API JSON; probability shown.	Matches expected in lab run (May 2026).	Pass
TC-13	REQ-13	Upload multiple images where enabled; verify batch handling.	Multiple results returned or per-file errors isolated.	Matches expected in lab run (May 2026).	Pass
TC-14	REQ-14	Navigate away and back; confirm layout consistency.	Styling remains consistent across navigation.	Matches expected in lab run (May 2026).	Pass

Test ID	REQ	Description / Steps	Expected	Actual	Status
TC-15	REQ-15	Observe survey progress indicator across steps.	Progress indicator updates monotonically with steps.	Matches expected in lab run (May 2026).	Pass
TC-16	REQ-16	Open itinerary route directly without state.	Guided empty-state directs user to quiz flow.	Matches expected in lab run (May 2026).	Pass

Table 6.2: Non-functional requirement verification matrix (representative)

Test ID	REQ	Description / Method	Expected	Actual	Status
TC-17	REQ-17	Load primary pages; observe interactive readiness using browser performance tooling.	Shell usable within agreed demo threshold.	Matches expected in lab run (May 2026).	Pass
TC-18	REQ-18	Build frontend static assets; verify separation from backend process.	Distinct client/server artifacts produced.	Matches expected in lab run (May 2026).	Pass
TC-19	REQ-19	Exercise each JSON endpoint with valid/invalid payloads; verify response schema.	Consistent JSON bodies; HTTP codes align with errors.	Matches expected in lab run (May 2026).	Pass
TC-20	REQ-20	Send malformed arrays/bounds (wrong answer count, out-of-range days).	Validation rejects before model invocation.	Matches expected in lab run (May 2026).	Pass
TC-21	REQ-21	Temporarily rename itinerary CSV; call endpoint.	Service-unavailable style error surfaced clearly.	Matches expected in lab run (May 2026).	Pass

Test ID	REQ	Description / Method	Expected	Actual	Status
TC-22	REQ-22	Review upload persistence behaviour vs. coursework policy.	No unintended long-term storage for prototype scope.	Matches expected in lab run (May 2026).	Pass
TC-23	REQ-23	Request itinerary for a city absent in CSV (if test harness permits).	Error explains missing attraction data.	Matches expected in lab run (May 2026).	Pass
TC-24	REQ-24	Repository inspection: module boundaries and asset locations.	Clear separation across frontend/backend/ML trees.	Matches expected in lab run (May 2026).	Pass
TC-25	REQ-25	Follow README runbooks on a clean machine checklist.	Documented steps sufficient for teaching assistant reproduction.	Matches expected in lab run (May 2026).	Pass
TC-26	REQ-26	Start backend; verify model load log lines appear.	Console diagnostics aid troubleshooting.	Matches expected in lab run (May 2026).	Pass
TC-27	REQ-27	Tab through quiz controls; verify visible focus indicators.	Keyboard operable baseline confirmed for demo.	Matches expected in lab run (May 2026).	Pass

Notes:

- Non-functional requirements REQ-17–REQ-27 may additionally require evidence attachments (logs, screenshots of network traces, or short screen recordings) per departmental rubric.
- For external submission, attach screenshots correlating TC-01–TC-16 with Figure 5.1–Figure 5.5 placeholders replaced by real captures.

Chapter 7

Contribution of Team Members

Team Member	Contribution (representative split)
Mukul Aggarwal	Backend Flask integration, CNN inference route validation, deployment scripts, API documentation updates.
Dhvani JOneja	Frontend trip planner UX, route/state handling, UI styling alignment, user-flow testing checklist.
Swasti Garg	ML pipeline alignment (Model1/Model2/Model3), dataset verification for itinerary CSV, recommender smoke tests.
Harsh Agrawal	Testing matrix maintenance, SRS/report LaTeX structure and diagrams, README and reproducibility packaging.

Table 7.1: Team contribution summary (high level).

Cross-cutting contributions: joint bug triage, demo rehearsal, dataset cleaning sessions, and integration testing nights preceding milestone reviews.

Chapter 8

References

[1] Wiegers, K. and Beatty, J. (2013). *Software Requirements*, 3rd ed. Microsoft Press. (Includes classic SRS guidance and requirement quality practices.)

[2] International Organization for Standardization / IEC / IEEE (2018). ISO/IEC/IEEE 29148: Systems and software engineering — Life cycle processes — Requirements engineering. `https://www.iso.org/standard/72038.html`

[3] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). UC Irvine.

[4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12, 2825–2830.

[5] Abadi, M. et al. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. (Software available from tensorflow.org).

[6] React Team (2026). React Documentation. `https://react.dev/`

[7] Vite Team (2026). Vite: Next Generation Frontend Tooling. `https://vite.dev/`

[8] Pallets Projects (2026). Flask Documentation. `https://flask.palletsprojects.com/`

Chapter 9

Standard SRS Structure (Karl Wiegers Template)

This chapter restates and completes the classic SRS skeleton recommended in widely used software requirements practice, aligned with Wiegers-style organization. Earlier chapters in this combined report provide project-specific evidence (problem framing, UML sketches, implementation narrative, and verification tables). This chapter consolidates the canonical structure for academic completeness.

9.1 Introduction

9.1.1 Purpose

This document specifies requirements for **Bon Voyage**, an academic prototype web application that combines machine-learning inference services (travel personality classification, destination recommendation, itinerary generation, and day/night photo classification) with a React-based user interface. The purpose is to enable a shared, reviewable contract among developers, evaluators, and stakeholders for demonstration and assessment.

9.1.2 Document Conventions

Requirements use unique identifiers REQ-n throughout functional and non-functional categories to preserve a single traceability namespace. The keyword conventions follow common practice: "shall" denotes mandatory behaviour for the scoped release unless explicitly marked as future work. Screen names use `/<route>` notation aligned with the React router implementation.

9.1.3 Intended Audience and Reading Suggestions

Developers should read Chapter 4–8 alongside REQ tables. **Evaluators** should prioritize verification traceability (Chapter 9) and operational reliability statements (REQ-17–REQ-27). **Product owners** should focus on Chapter 1 (customer problem decomposition) and Chapter 2 (business goals).

9.1.4 Product Scope

In-scope capabilities include: personality questionnaire and inference; city shortlisting; itinerary generation for a selected city; day/night classification API; and minimal deployment guidance for coursework labs. Out-of-scope for the current prototype: payments, production authentication/authorization, live hotel booking integrations, real-time traffic routing, multi-user collaborative trips, and mobile native applications.

9.1.5 References

See the numbered bibliography in Chapter 11. Additional internal references include repository documentation files packaged with the source distribution.

9.2 Overall Description

9.2.1 Product Perspective

Bon Voyage is a greenfield system that composes separate ML assets behind a single Flask façade. It does not assume an existing enterprise database; instead it ships with curated offline artifacts (pickled models, JSON vector bundles, and CSV attractions). The product integrates with standard browsers and uses HTTP JSON APIs suitable for academic demonstrations.

9.2.2 Product Functions

Major functions include: (1) survey-driven personality inference; (2) personality-conditioned city ranking; (3) constrained itinerary generation based on attraction metadata; (4) CNN-based classification of uploaded photos; and (5) static web UI orchestration of these services.

9.2.3 User Classes and Characteristics

- **Casual traveller (primary):** expects guided UX, limited configuration, fast feedback.
- **Evaluator/marker (secondary):** expects reproducibility, clear documentation, stable demo scripts.
- **Administrator/maintainer (limited):** updates datasets and model binaries between milestones.

9.2.4 Operating Environment

Client: modern evergreen browsers (Chrome/Edge/Firefox). Server: Python 3.x with scientific stack (NumPy, scikit-learn, TensorFlow) and OpenCV for image decode pipelines. Development: Node.js toolchain for frontend bundling. Deployment: suitable for localhost and LAN demos; production hardening is out of scope.

9.2.5 Design and Implementation Constraints

Constraints include: deterministic integer survey mapping for scaler compatibility; fixed maximum itinerary horizon to prevent runaway generation; reliance on curated CSV coverage for cities; and filesystem paths to ML assets relative to repository layout.

9.2.6 User Documentation

User-facing documentation is embedded in UI affordances (labels, progress indication, and error banners). Additional setup instructions are maintained in repository README files for backend

and frontend.

9.2.7 Assumptions and Dependencies

The system assumes attraction data completeness for demo cities; absence of data yields explicit failures. It depends on compatible Python package versions, availability of trained model binaries, and a running Flask process reachable from the web client (dev proxy or deployed origin).

9.3 External Interface Requirements

9.3.1 User Interfaces

The UI provides landing navigation, a stepped survey, results panels, itinerary visualization, and upload classification views. Visual branding uses a dark, high-contrast presentation intended for projector demonstrations.

9.3.2 Hardware Interfaces

No special hardware beyond standard keyboard/mouse/touch for web interaction. GPU acceleration is optional for TensorFlow but not required for demonstration-scale workloads in coursework settings.

9.3.3 Software Interfaces

The backend integrates TensorFlow for CNN inference, scikit-learn routines for vector scoring, and pandas for CSV ingestion. JSON is the interchange format for web APIs.

9.3.4 Communications Interfaces

The prototype uses HTTP/1.1 between browser and Flask. HTTPS termination would be provided externally in a production deployment (out of scope for default lab configuration).

9.4 System Features

9.4.1 Feature SF-A: Personality inference and city recommendation

Description and Priority: High. This feature underpins personalization.
Stimulus/Response Sequences:
Stimulus: user submits 12 answers.
Response: server returns predicted personality label; client requests recommended cities and displays up to five names.
Functional Requirements: REQ-2, REQ-3, REQ-4, REQ-5.

9.4.2 Feature SF-B: Itinerary generation for a chosen destination

Description and Priority: High. Converts selection into an actionable time-bucketed plan.
Stimulus/Response Sequences:

Stimulus: user selects city, sets duration within bounds, and triggers generation/refresh.
Response: server returns day-keyed itinerary structures; client renders schedule.
Functional Requirements: REQ-6, REQ-7, REQ-8, REQ-9.

9.5 Other Nonfunctional Requirements

Performance, maintainability, security posture for coursework deployments, and robust error reporting are enumerated as REQ-17–REQ-27 to maintain one consolidated requirement list aligned with the project's test matrix policy.

9.6 Other Requirements

Additional institution-specific rubric items (demonstration checklist, plagiarism policy adherence, dataset licensing statements for course submissions) should be appended by the course staff if required. No further vendor-specific certifications are claimed for this prototype.

Appendix A

Glossary

API Application Programming Interface; here, JSON-over-HTTP endpoints exposed by Flask.

CNN
: Convolutional Neural Network used for day/night image discrimination.

Itinerary
: A structured mapping from trip day indices to ordered activities with timestamps and labels.

Likert scale
: Ordinal response scale; Bon Voyage maps UI selections to integers compatible with training assumptions.

SVM
: Support Vector Machine classifier used for travel personality labeling in Model1.

Traceability
: Mapping between requirements, design artifacts, and test evidence.

Appendix B

Analysis Models

This appendix references the analysis artifacts presented as diagrams within the main report: Figure 4.1 (conceptual classes/tiers), Figure 3.1 (use cases), Figure 4.2 (activity), and Figure 4.3 (sequence). For submission hardening, teams may include additional data-flow diagrams showing dataset ingestion and preprocessing for Model2 and Model3.

Appendix C

To Be Determined List

- **TBD-1:** Final institution name formatting and cover-page approval stamp (if mandated).
- **TBD-2:** Production hosting choice (cloud provider, SSL policy) beyond coursework localhost scope.
- **TBD-3:** Expanded attraction dataset coverage beyond demonstration cities (ongoing curation).
- **TBD-4:** Formal accessibility audit (WCAG) beyond baseline keyboard labeling noted in REQ-27.
- **TBD-5:** Automated end-to-end testing harness (Playwright/Cypress) for regression suites in CI.